

リバーシ コード全体

1. main.dart

```
1 // main.dart
2 import 'package:flutter/material.dart'; // FlutterのUI機能を利用するためのラ
   イフラリ
3 import 'logic.dart'; // リバーシのゲームロジックを定義したファイル
4 import 'package:url_launcher/url_launcher.dart'; // 外部ブラウザでWebページ
   を開くためのライブラリ
5
6 // アプリケーションの開始地点
7 void main() {
8   runApp(const MyApp()); // MyAppウィジェットを起動
9 }
10
11 // アプリ全体を管理するクラス
12 class MyApp extends StatelessWidget {
13   const MyApp({super.key});
14
15   @override
16   Widget build(BuildContext context) {
17     return MaterialApp(
18       title: 'Reversi', // アプリ名
19       // アプリ全体のテーマ設定
20       theme: ThemeData(
21         colorScheme: ColorScheme.fromSeed(
22           seedColor: Colors.green, // メインカラーを緑に設定
23         ),
24         useMaterial3: true, // Material Design 3を使用
25       ),
26       home: const ReversiPage(), // 起動時に表示する画面
27     );
28   }
29 }
30
31 // リバーシ画面を表すStatefulWidget
32 // ゲームの進行に応じて画面を更新するためStatefulWidgetを使用
33 class ReversiPage extends StatefulWidget {
34   const ReversiPage({super.key});
35
36   @override
37   State<ReversiPage> createState() => _ReversiPageState(); // 状態管理クラ
   スを生成
38 }
39
40 // ReversiPageの状態管理クラス
41 class _ReversiPageState extends State<ReversiPage> {
42   final ReversiGame _game = ReversiGame(); // リバーシのゲームロジックを管理す
   るオブジェクト
43
44   bool _isAiThinking = false; // AI思考中かどうかを管理するフラグ
45
46   // AIの手番になった際に自動で着手を行う関数
47   void _triggerAiMoveIfNeeded() async {
48     // AI対戦モード中かつゲームが終了していない場合のみ実行
49     if (_game.gameMode != 0 && !_game.isGameOver()) {
50       // 現在の手番がAIでなければ処理終了
```

```

52     if (_game.currentPlayer != _game.aiPlayer) return;
53
54     setState(() {
55         _isAiThinking = true; // AI思考中状態にする
56     });
57
58     // AIの思考演出として1秒待機
59     Future.delayed(const Duration(milliseconds: 1000), () {
60         // ウィジェットが破棄されていた場合は処理終了
61         if (!mounted) return;
62
63         List<int>? aiMove; // AIが選択した手を保存
64
65         // 弱AI
66         if (_game.gameMode == 1) {
67             aiMove = _game.getWeakAiMove();
68
69             // 強AI
70         } else if (_game.gameMode == 2) {
71             aiMove = _game.getStrongAiMove();
72
73             // 最強AI
74         } else if (_game.gameMode == 3) {
75             aiMove = _game.getExpertAiMove();
76         }
77
78         // AIが有効な手を選択した場合
79         if (aiMove != null) {
80             _game.playTurn(
81                 aiMove[0], // 行番号
82                 aiMove[1], // 列番号
83             );
84         }
85
86         setState(() {
87             _isAiThinking = false; // 思考終了
88         });
89
90         // パスなどでAIの手番が続く場合に再度実行
91         _triggerAiMoveIfNeeded();
92     });
93 }
94 }
95
96 @override
97 Widget build(BuildContext context) {
98     // 現在の手番に応じて背景色を変更
99     final Color backgroundColor = _game.currentPlayer == 1
100         ? Colors
101             .green
102             .shade50 // 黒の手番
103         : Colors.blue.shade50; // 白の手番
104
105     return Scaffold(
106         // アプリ上部のヘッダー
107         appBar: AppBar(
108             title: const Text('リバーシ'), // タイトル表示
109             centerTitle: true, // タイトルを中央揃え
110             backgroundColor: Colors.transparent, // 背景透明

```

```

111     ),
112
113     // 背景色をアニメーション付きで変更
114     body: AnimatedContainer(
115       duration: const Duration(milliseconds: 500), // アニメーション時間
116       curve: Curves.easeInOut, // 緩やかな変化
117       color: backgroundColor, // 手番に応じた背景色
118       // スマートフォンのノッチ等と重ならない領域
119       child: SafeArea(
120         child: Column(
121           children: [
122             const SizedBox(height: 12), // 上部余白
123
124             _buildScoreBoard(), // スコア・操作パネル表示
125
126             const SizedBox(height: 12), // 間隔
127
128             Expanded(child: Center(child: _buildBoard()))),
129           1,
130         ),
131       ),
132     ),
133   );
134 }
135
136 // スコアボードおよびゲーム状態表示エリアを生成
137 Widget _buildScoreBoard() {
138   // 現在の手番に応じて石の色を決定
139   final Color turnColor = _game.currentPlayer == 1
140     ? Colors.black
141     : Colors.white;
142
143   // 現在の手番プレイヤー名を表示するテキスト
144   String turnText = _game.currentPlayer == 1
145     ? "${_game.blackPlayerName} (黒) の番"
146     : "${_game.whitePlayerName} (白) の番";
147
148   // AI思考中は専用メッセージを表示
149   if (_isAiThinking) {
150     turnText = "AI思考中...";
151   }
152
153   // ゲーム終了判定
154   final bool isOver = _game.isGameOver();
155
156   // 黒石の数を取得
157   int black = _game.countStones(1);
158
159   // 白石の数を取得
160   int white = _game.countStones(2);
161
162   // 石数を比較して勝敗メッセージを決定
163   String resultText = black > white
164     ? "🏆 黒の勝利！"
165     : white > black
166     ? "🏆 白の勝利！"
167     : "👉 引き分け！";
168

```

```

169 // 勝者に応じて結果表示エリアの背景色を変更
170 Color winnerContainerColor = black > white
171     ? Colors.black
172     : white > black
173     ? Colors.white
174     : Colors.grey.shade300;
175
176 // 勝敗表示の文字色を設定
177 Color winnerTextColor = (black > white) ? Colors.white :
Colors.black;
178
179 return Column(
180     children: [
181         // 手番表示と勝敗表示を行うエリア
182         AnimatedContainer(
183     メーション時間 duration: const Duration(milliseconds: 300), // 表示切替時のアニメーション時間
184         // ゲーム中と終了時で余白サイズを変更
185         padding: EdgeInsets.symmetric(
186             horizontal: isOver ? 32 : 24,
187             vertical: isOver ? 14 : 8,
188         ),
189
190         decoration: BoxDecoration(
191             // ゲーム終了時は勝者色, 通常時は半透明の白背景
192             color: isOver
193                 ? winnerContainerColor
194                 : Colors.white.withValues(alpha: 0.7),
195             borderRadius: BorderRadius.circular(30), // 角を丸くする
196             boxShadow: // ゲーム終了時は影を強調
197                 isOver
198                     ? [
199                         BoxShadow(
200                             color: Colors.black26,
201                             blurRadius: 10, // 影のぼかし量
202                             offset: const Offset(0, 4), // 影の位置
203                         ),
204                     ]
205                     : const [
206                         BoxShadow(
207                             color: Colors.black12,
208                             blurRadius: 4,
209                             offset: Offset(0, 2),
210                         ),
211                     ],
212         ),
213
214         // 横方向に要素を配置
215         child: Row(
216             mainAxisAlignment: MainAxisAlignment.min,
217             children: [
218                 // ゲーム進行中のみ現在の手番の石を表示
219                 if (!isOver) ...[
220                     AnimatedContainer(
221     アニメーション duration: const Duration(milliseconds: 200), // 石の切替
222
223                     width: 24, // 石の幅

```

```

224         height: 24, // 石の高さ
225
226         decoration: BoxDecoration(
227             shape: BoxShape.circle, // 円形
228
229             color: turnColor, // 黒番なら黒, 白番なら白
230
231             border: Border.all(
232                 // 白石の場合のみ外枠を表示
233                 color: _game.currentPlayer == 1
234                     ? Colors.transparent
235                     : Colors.black26,
236
237             width: 2,
238         ),
239     ),
240 ),
241
242     const SizedBox(width: 12), // 石と文字の間隔
243 ],
244
245 // 手番または勝敗結果を表示
246 Text(
247     // ゲーム中は現在の手番, 終了時は勝敗結果を表示
248     isOver ? resultText : turnText,
249
250     style: TextStyle(
251         // 終了時は文字サイズを大きくして強調
252         fontSize: isOver ? 22 : 18,
253
254         fontWeight: FontWeight.bold, // 太字
255         // 勝敗表示時は背景色に応じて文字色を変更
256         color: isOver ? winnerTextColor : Colors.black,
257     ),
258 ),
259 1,
260 ),
261 ),
262
263 // パス発生時のメッセージを表示
264 // ゲーム終了後は表示しない
265 if (_game.passMessage.isNotEmpty && !isOver)
266     Padding(
267         // 上方向に少し余白を追加
268         padding: const EdgeInsets.only(top: 4),
269
270         child: Text(
271             // 「黒は置けないためパス」などを表示
272             _game.passMessage,
273
274             style: const TextStyle(
275                 color: Colors.red, // 注意表示のため赤色
276                 fontSize: 14,
277                 fontWeight: FontWeight.bold,
278             ),
279         ),
280     ),
281
282     const SizedBox(height: 8), // 間隔

```

```

283 // 黒石と白石の数を表示するエリア
284 Row(
285   mainAxisAlignment: MainAxisAlignment.center,
286
287   children: [
288     // 黒石数表示
289     _buildSmallScoreBadge(player: 1, count: black),
290
291     const SizedBox(width: 20),
292
293     // 白石数表示
294     _buildSmallScoreBadge(player: 2, count: white),
295   ],
296 ),
297
298 const SizedBox(height: 12), // 間隔
299 // 操作パネル
300 // 戻るボタン・リセットボタン・ゲームモード切替を配置
301 Padding(
302   padding: const EdgeInsets.symmetric(horizontal: 16.0),
303
304   child: Wrap(
305     // ボタン同士の間隔
306     spacing: 8,
307     runSpacing: 8,
308
309     alignment: WrapAlignment.center,
310     crossAxisAlignment: WrapCrossAlignment.center,
311
312     children: [
313       // 解説ページへ戻るボタン
314       IconButton(
315         onPressed: () async {
316           // 解説ページのURLを生成
317           final Uri url = Uri.parse(
318             'https://www.oit.ac.jp/labs/rd/rssrv/kobayashi-lab/~
319             takamiya/flutter/reversi.html',
320           );
321
322           // URLが開ける場合のみブラウザ起動
323           if (await canLaunchUrl(url)) {
324             await launchUrl(url, mode:
325               LaunchMode.externalApplication);
326           }
327         },
328
329         icon: const Icon(Icons.logout), // 戻るアイコン
330         tooltip: '解説ページへ戻る',
331       ),
332
333       // ゲームリセットボタン
334       ElevatedButton.icon(
335         onPressed: () {
336           setState(() {
337             // ゲーム状態を初期化
338             _game.reset();
339
340             // AI思考状態も解除
341             _isAiThinking = false;

```

```

340         });
341
342         // AI戦の場合は初手AIを実行
343         _triggerAiMoveIfNeeded();
344     },
345
346     icon: const Icon(Icons.refresh), // リセットアイコン
347     label: const Text("リセット"),
348 ),
349
350 // ゲームモード切替ボタン群
351 ToggleButtons(
352     // 現在選択中のモードを表示
353     isSelected: [
354         _game.gameMode == 0, // 二人対戦
355         _game.gameMode == 1, // AI弱
356         _game.gameMode == 2, // AI強
357         _game.gameMode == 3, // AI最強
358     ],
359
360     // ボタン押下時の処理
361     onPressed: (int index) {
362         setState(() {
363             // 選択されたモードに変更
364             _game.gameMode = index;
365
366             // 新しいモードでゲーム開始
367             _game.reset();
368
369             _isAiThinking = false;
370         });
371
372         // AI戦の場合はAI着手判定
373         _triggerAiMoveIfNeeded();
374     },
375
376     borderRadius: BorderRadius.circular(20), // ボタン角丸
377
378     constraints: const BoxConstraints(minHeight: 36,
379         minWidth: 68),
380
381     selectedColor: Colors.white, // 選択時文字色
382
383     fillColor: Colors.green.shade700, // 選択時背景色
384
385     children: const [
386         // 人対人モード
387         Text(
388             "二人対戦",
389             style: TextStyle(fontSize: 12, fontWeight:
390                 FontWeight.bold),
391         ),
392         // ランダムAI
393         Text(
394             "AI (弱) ",
395             style: TextStyle(fontSize: 12, fontWeight:
396                 FontWeight.bold),
397         ),

```

```

396
397         // 重み付けAI
398         Text(
399             "AI (強) ",
400             style: TextStyle(fontSize: 12, fontWeight:
FontWeight.bold),
401         ),
402
403         // 探索AI
404         Text(
405             "AI (最強) ",
406             style: TextStyle(fontSize: 11, fontWeight:
FontWeight.bold),
407         ),
408     ],
409 ),
410 ],
411 ),
412 ),
413 ],
414 );
415 }
416
417 // 石の色と個数を表示するスコアバッジを生成
418 Widget _buildSmallScoreBadge({required int player, required int count})
419 {
420     return Row(
421         children: [
422             // 石を表す円
423             Container(
424                 width: 16,
425                 height: 16,
426                 decoration: BoxDecoration(
427                     shape: BoxShape.circle,
428
429                     // プレイヤーに応じて石の色を変更
430                     color: player == 1 ? Colors.black : Colors.white,
431
432                     border: Border.all(color: Colors.black38, width: 1),
433                 ),
434             ),
435
436             const SizedBox(width: 6), // 石と個数表示の間隔
437             // 石の個数を表示
438             Text(
439                 "$count個",
440                 style: const TextStyle(fontSize: 14, fontWeight:
FontWeight.bold),
441             ),
442         ],
443     );
444 }
445
446 // リバーシ盤面を生成
447 Widget _buildBoard() {
448     return Padding(
449         // 盤面の外側余白

```

```

450 padding: const EdgeInsets.all(16.0),
451
452 child: AspectRatio(
453   // 常に正方形の盤面を維持
454   aspectRatio: 1,
455
456   child: Container(
457     // 盤面全体の外枠
458     decoration: BoxDecoration(
459       border: Border.all(color: Colors.black, width: 2),
460     ),
461
462     // 8×8のマスを生成
463     child: GridView.builder(
464       // スクロールを無効化
465       physics: const NeverScrollableScrollPhysics(),
466
467       gridDelegate: const
SliverGridDelegateWithFixedCrossAxisCount(
468         // 8列構成
469         crossAxisCount: 8,
470       ),
471
472       itemCount: 64, // 8×8=64マス
473
474       itemBuilder: (context, index) {
475         // 1次元インデックスを行番号へ変換
476         int row = index ~/ 8;
477
478         // 1次元インデックスを列番号へ変換
479         int col = index % 8;
480
481         // 現在マスの状態取得
482         // 0:空き 1:黒 2:白
483         int stoneState = _game.board[row][col];
484
485         return GestureDetector(
486           // マスがタップされた時の処理
487           onTap: () {
488             // AI思考中または石が存在する場合は無効
489             if (_isAiThinking || stoneState != 0) {
490               return;
491             }
492
493             // 石を置けた場合
494             if (_game.playTurn(row, col)) {
495               // 画面更新
496               setState(() {});
497
498               // AI対戦時はAI着手判定
499               _triggerAiMoveIfNeeded();
500             }
501           },
502
503           child: Container(
504             // マスの見た目設定
505             decoration: BoxDecoration(
506               // リバーシ盤面の緑色
507               color: const Color(0xff0f7d32),

```

```

508
509 // マスの境界線
510 border: Border.all(color: Colors.black54, width:
0.5),
511 ),
512
513 child: Stack(
514   alignment: Alignment.center,
515
516   children: [
517     // 石の描画
518     ReversiStone(state: stoneState),
519
520     // 合法手の場所に黄色マーカを表示
521     if (stoneState == 0 &&
_game.canPlaceStone(row, col,
_game.currentPlayer))
522       const CircleAvatar(
523         radius: 5,
524         backgroundColor: Colors.yellow,
525       ),
526     ],
527   ),
528 ),
529 );
530 },
531 ),
532 ),
533 ),
534 ),
535 );
536 }
537 }
538
539 // リバーシの石を描画するウィジェット
540 class ReversiStone extends StatelessWidget {
541   // 石の状態 0:空きマス 1:黒石 2:白石
542   final int state;
543   const ReversiStone({super.key, required this.state});
544
545   @override
546   Widget build(BuildContext context) {
547     if (state == 0) return const SizedBox(); // 空きマスの場合は何も表示しない
548
549     final double turns = state == 1 ? 0.0 : 0.5; // 黒石なら0回転, 白石なら
180度回転
550
551     return FractionallySizedBox(
552       // マスの85%のサイズで石を描画
553       widthFactor: 0.85,
554       heightFactor: 0.85,
555
556       child: AnimatedRotation(
557         // 石の状態に応じて回転
558         turns: turns,
559
560         // 回転アニメーション時間
561         duration: const Duration(milliseconds: 300),
562

```

```

563 // 滑らかな回転を実現
564 curve: Curves.easeInOut,
565
566 child: Stack(
567   children: [
568     // 白石の描画
569     // 裏面として利用
570     Transform.flip(
571       // 上下反転
572       flipY: true,
573
574       child: _buildStoneContainer(
575         colors: [Colors.white, const Color(0xffdddddd)],
576         isBlack: false,
577       ),
578     ),
579     // 黒石の描画
580     // stateに応じて表示・非表示を切替
581     Opacity(
582       // 黒石なら表示, 白石なら非表示
583       opacity: state == 1 ? 1.0 : 0.0,
584
585       child: _buildStoneContainer(
586         colors: [const Color(0xff555555), Colors.black],
587         isBlack: true,
588       ),
589     ),
590   ],
591   1,
592 ),
593 );
594 };
595 }
596
597 // 石本体の見た目を生成する関数
598 Widget _buildStoneContainer({
599   required List<Color> colors,
600   required bool isBlack,
601 }) {
602   return Container(
603     decoration: BoxDecoration(
604       // 円形の石を生成
605       shape: BoxShape.circle,
606
607       // 立体感を出すための影
608       boxShadow: const [
609         BoxShadow(color: Colors.black38, blurRadius: 4, offset:
610         Offset(2, 2)),
611       ],
612
613       // グラデーションによる光沢表現
614       gradient: LinearGradient(
615         begin: Alignment.topLeft,
616         end: Alignment.bottomRight,
617         colors: colors,
618       ),
619     );

```

```
620     }  
621 }
```

2. logic.dart

```
1  import 'dart:math'; // 数学関数や乱数生成機能を利用するためのライブラリ
2
3  /// 置換表(Transposition Table)に保存するデータ構造
4  class TTEEntry {
5      final int hash; // 盤面のハッシュ値
6      final int score; // その盤面の評価値
7      final int depth; // 探索した深さ
8      final int flag; // 評価値の種類 0:正確値 1:下限値(Betaカット) 2:上限値
9      final List<int>? bestMove; // その盤面で最善と判断された手
10     // コンストラクタ
11     TTEEntry({
12         required this.hash,
13         required this.score,
14         required this.depth,
15         required this.flag,
16         this.bestMove,
17     });
18 }
19
20 // リバーシゲーム全体を管理するクラス
21 class ReversiGame {
22     late List<List<int>> board; // 盤面情報 0:空き 1:黒 2:白
23     int currentPlayer = 1; // 現在の手番 1=黒, 2=白
24     String passMessage = ""; // パス発生時の表示メッセージ
25     int gameMode = 0; // 0:二人対戦 1:弱AI 2:強AI 3:最強AI
26     int aiPlayer = 2; // AIが担当するプレイヤー番号
27
28     // 黒プレイヤー名を取得
29     String get blackPlayerName =>
30         gameMode == 0 ? "プレイヤー1" : (aiPlayer == 1 ? "AI" : "あなた");
31
32     // 白プレイヤー名を取得
33     String get whitePlayerName =>
34         gameMode == 0 ? "プレイヤー2" : (aiPlayer == 2 ? "AI" : "あなた");
35
36     // 8方向探索用ベクトル
37     final List<List<int>> _directions = [
38         [-1, -1], // 左上
39         [-1, 0], // 上
40         [-1, 1], // 右上
41         [0, -1], // 左
42         [0, 1], // 右
43         [1, -1], // 左下
44         [1, 0], // 下
45         [1, 1], // 右下
46     ];
47
48     final Map<int, TTEEntry> _transpositionTable = {}; // 置換表 同じ盤面の再
    探索を防ぐために使用
49     late List<List<int>> _zobristTable; // Zobrist Hash用乱数テーブル [64マ
    ス][空き・黒・白]
50     int _zobristPlayerXor = 0; // 手番情報をハッシュへ反映するためのXOR値
51     int _currentHash = 0; // 現在盤面のハッシュ値
52
53     // Killer Heuristic用テーブル [探索深さ][最大2手]
54     late List<List<List<int>?>> _killerMoves;
```

```

55 late List<List<int>> _historyTable; // History Heuristic用テーブル 各手の
有効度を保存
56
57 // Edge+2 Pattern Evaluation用評価テーブル 3^10 = 59049通りのパターンを保存
58 static final List<int> _edgePatternTable = List.filled(59049, 0);
59 static bool _patternTableInitialized = false; // 評価テーブル初期化済み判
定
60
61 // コンストラクタ
62 ReversiGame() {
63     _initZobrist(); // Zobrist Hashテーブル生成
64     _initPatternTable(); // パターン評価テーブル生成
65     reset(); // ゲーム初期化
66 }
67
68 // Zobrist Hash用の乱数テーブルを生成
69 void _initZobrist() {
70     final rand = Random(42); // 再現性を確保するため固定シードを使用
71
72     // 64マス × 3状態(空・黒・白)の乱数を生成
73     _zobristTable = List.generate(
74         64,
75         (_) => List.generate(3, (_) => rand.nextInt(0x7fffffff)),
76     );
77
78     _zobristPlayerXor = rand.nextInt(0x7fffffff); // 手番情報用の乱数を生成
79 }
80
81 // パターン評価テーブルの初期化
82 void _initPatternTable() {
83     // すでに初期化済みなら終了
84     if (_patternTableInitialized) {
85         return;
86     }
87
88     // 3^10通りの全パターンを生成
89     for (int i = 0; i < 59049; i++) {
90         int score = 0; // 評価値
91         int tmp = i; // パターン番号を保持
92         List<int> p = List.filled(10, 0); // 10マス分の状態を格納
93
94         // 10桁の3進数へ変換
95         for (int j = 0; j < 10; j++) {
96             p[j] = tmp % 3;
97             tmp ~/= 3;
98         }
99
100        // 左端角の評価
101        if (p[0] == 1) {
102            score += 300;
103        }
104
105        if (p[0] == 2) {
106            score -= 300;
107        }
108
109        // 右端角の評価
110        if (p[7] == 1) {
111            score += 300;
112        }

```

```

113
114     if (p[7] == 2) {
115         score -= 300;
116     }
117
118     // 左角が空いている場合のXマス評価
119     if (p[0] == 0) {
120         if (p[8] == 1) {
121             score -= 150;
122         }
123
124         if (p[8] == 2) {
125             score += 150;
126         }
127     }
128
129     // 右角が空いている場合のXマス評価
130     if (p[7] == 0) {
131         if (p[9] == 1) {
132             score -= 150;
133         }
134
135         if (p[9] == 2) {
136             score += 150;
137         }
138     }
139
140     // 辺上の通常マス評価
141     for (int j = 1; j < 7; j++) {
142         if (p[j] == 1) {
143             score += 20;
144         }
145
146         if (p[j] == 2) {
147             score -= 20;
148         }
149     }
150
151     _edgePatternTable[i] = score; // 計算した評価値を保存
152 }
153
154 _patternTableInitialized = true; // 初期化完了フラグ
155 }
156
157 // ゲームを初期状態へ戻す
158 void reset() {
159     board = List.generate(8, (_) => List.generate(8, (_) => 0)); // 8×8
160     // 空盤面を生成
161     board[3][3] = 2; // 初期配置 (白)
162     board[3][4] = 1; // 初期配置 (黒)
163     board[4][3] = 1; // 初期配置 (黒)
164     board[4][4] = 2; // 初期配置 (白)
165
166     currentPlayer = 1; // 黒から開始
167     passMessage = ""; // パスメッセージを初期化
168     _transpositionTable.clear(); // 置換表をクリア
169
170     // Killer Heuristicテーブルを初期化
171     _killerMoves = List.generate(60, (_) => List.filled(2, null));

```

```

171 // History Heuristicテーブルを初期化
172 _historyTable = List.generate(8, (_) => List.filled(8, 0));
173
174 _calculateFullHash(); // 現在盤面のハッシュ値を計算
175
176 final List<int> roles = [1, 2]; // AI先手・後手をランダム決定
177
178 roles.shuffle();
179
180 aiPlayer = roles.first;
181 }
182
183 // 現在の盤面からハッシュ値を再計算
184 void _calculateFullHash() {
185     _currentHash = 0; // ハッシュ値初期化
186
187     // 全マスを走査
188     for (int r = 0; r < 8; r++) {
189         for (int c = 0; c < 8; c++) {
190             int state = board[r][c]; // マスの状態取得
191
192             シュ生成 _currentHash ^= _zobristTable[r * 8 + c][state]; // XOR演算でハッ
193         }
194     }
195
196     // 黒番の場合は手番情報も反映
197     if (currentPlayer == 1) {
198         _currentHash ^= _zobristPlayerXor;
199     }
200 }
201
202 // 相手プレイヤー番号を取得
203 int getOpponent(int player) => player == 1 ? 2 : 1;
204
205 // 指定座標が盤面内か判定
206 bool _isInside(int r, int c) => r >= 0 && r < 8 && c >= 0 && c < 8;
207
208 // 指定位置へ石を置けるか判定
209 bool canPlaceStone(int row, int col, int player) {
210     // すでに石がある場合は置けない
211     if (board[row][col] != 0) {
212         return false;
213     }
214
215     int opponent = getOpponent(player); // 相手プレイヤー取得
216
217     // 8方向を探索
218     for (var dir in _directions) {
219         int r = row + dir[0];
220         int c = col + dir[1];
221
222         bool foundOpponent = false; // 相手石を見つけたか判定
223
224         // 盤面外へ出るまで探索
225         while (_isInside(r, c)) {
226             // 相手石発見
227             if (board[r][c] == opponent) {
228                 foundOpponent = true;
229             }

```

```

230         // 自分の石発見
231         else if (board[r][c] == player) {
232             // 間に相手石が存在するなら合法手
233             if (foundOpponent) {
234                 return true;
235             }
236
237             break;
238         }
239         // 空マスなら探索終了
240         else {
241             break;
242         }
243
244         // 次マスへ移動
245         r += dir[0];
246         c += dir[1];
247     }
248 }
249
250 return false; // どの方向にも挟めない場合
251 }
252
253 // 指定位置から石を反転する
254 void flipStones(int row, int col, int player) {
255     int opponent = getOpponent(player); // 相手プレイヤー取得
256
257     // 8方向を探索
258     for (var dir in _directions) {
259         List<List<int>> stonesToFlip = []; // 反転候補の石を保存
260
261         int r = row + dir[0];
262         int c = col + dir[1];
263
264         // 盤面内を探索
265         while (_isInside(r, c)) {
266             // 相手石なら候補へ追加
267             if (board[r][c] == opponent) {
268                 stonesToFlip.add([r, c]);
269             }
270             // 自分の石に到達した場合
271             else if (board[r][c] == player) {
272                 // 候補石をすべて反転
273                 for (var stone in stonesToFlip) {
274                     int idx = stone[0] * 8 + stone[1]; // ハッシュ値更新用インデック
275                     _currentHash ^= _zobristTable[idx][opponent]; // 相手石状態を
276                     _currentHash ^= _zobristTable[idx][player]; // 自分石状態を追加
277                     board[stone[0]][stone[1]] = player; // 石を反転
278                 }
279                 break;
280             }
281             // 空マスなら終了
282             else {
283                 break;
284             }
285
286             // 次マスへ移動

```

```

287         r += dir[0];
288         c += dir[1];
289     }
290 }
291 }
292
293 // 指定プレイヤーに合法手が存在するか判定
294 bool isValidMove(int player) {
295     // 全マスを探査
296     for (int r = 0; r < 8; r++) {
297         for (int c = 0; c < 8; c++) {
298             // 合法手を発見
299             if (canPlaceStone(r, c, player)) {
300                 return true;
301             }
302         }
303     }
304
305     // 合法手なし
306     return false;
307 }
308
309 // 指定プレイヤーの石数を取得
310 int countStones(int player) {
311     return board.expand((row) => row).where((cell) => cell ==
player).length;
312 }
313
314 // 両プレイヤーが置けない場合ゲーム終了
315 bool isGameOver() => !isValidMove(1) && !isValidMove(2);
316
317 // 1ターン分の着手処理
318 bool playTurn(int row, int col) {
319     // 合法手でない場合
320     if (!canPlaceStone(row, col, currentPlayer)) {
321         return false;
322     }
323
324     int idx = row * 8 + col; // 配置位置のインデックス
325     _currentHash ^= _zobristTable[idx][0]; // 空状態を削除
326     _currentHash ^= _zobristTable[idx][currentPlayer]; // 現在プレイヤー状
態を追加
327     board[row][col] = currentPlayer; // 石配置
328     flipStones(row, col, currentPlayer); // 石反転
329     int nextPlayer = getOpponent(currentPlayer); // 次プレイヤー取得
330     _currentHash ^= _zobristPlayerXor; // 手番変更をハッシュへ反映
331
332     // 相手が着手可能な場合
333     if (isValidMove(nextPlayer)) {
334         currentPlayer = nextPlayer;
335
336         passMessage = "";
337     }
338     // 相手が着手できない場合
339     else {
340         // 現在プレイヤーが置ける場合
341         if (isValidMove(currentPlayer)) {
342             // 手番変更を取り消す

```

```

343         _currentHash ^= _zobristPlayerXor;
344
345         passMessage = nextPlayer == 1 ? "黒は置けないためパス" : "白は置けな
いたためパス";
346     }
347     // 両者とも置けない場合
348     else {
349         passMessage = "両者とも置けないためゲーム終了";
350     }
351 }
352
353 return true;
354 }
355
356 // 指定プレイヤーの合法手一覧を生成
357 List<List<int>> _generateValidMoves(int player) {
358     List<List<int>> moves = [];
359
360     // 全マスを探査
361     for (int r = 0; r < 8; r++) {
362         for (int c = 0; c < 8; c++) {
363             // 合法手を追加
364             if (canPlaceStone(r, c, player)) {
365                 moves.add([r, c]);
366             }
367         }
368     }
369
370     return moves;
371 }
372
373 // 弱AI：合法手の中からランダムに選択
374 List<int>? getWeakAiMove() {
375     // 現在の手番プレイヤーの合法手一覧を取得
376     List<List<int>> validMoves = _generateValidMoves(currentPlayer);
377
378     // 合法手がなければパス
379     if (validMoves.isEmpty) {
380         return null;
381     }
382
383     validMoves.shuffle(); // 合法手をランダムに並び替え
384
385     return validMoves.first; // 先頭の手を返す
386 }
387
388 // 強AI：盤面評価値が最も高いマスを選択
389 List<int>? getStrongAiMove() {
390     // 現在の合法手を取得
391     List<List<int>> validMoves = _generateValidMoves(currentPlayer);
392
393     // 合法手がなければパス
394     if (validMoves.isEmpty) {
395         return null;
396     }
397
398     List<int> best = validMoves.first; // とりあえず最初の手を候補にする
399     int maxW = -999; // 最大評価値を保存する変数
400

```

```

401 // 全ての合法手を調査
402 for (var m in validMoves) {
403     int w = _getStaticWeight(m[0], m[1]); // そのマスの静的評価値を取得
404
405     // 現在の最大値より高ければ更新
406     if (w > maxW) {
407         maxW = w;
408         best = m;
409     }
410 }
411
412 return best; // 最も評価値の高い手を返す
413 }
414
415 // 🧠 最強AI : NegaScout探索を用いた高性能AI
416 List<int>? getExpertAiMove() {
417     // 現在の合法手を取得
418     List<List<int>> validMoves = _generateValidMoves(currentPlayer);
419
420     // 合法手がなければパス
421     if (validMoves.isEmpty) {
422         return null;
423     }
424
425     // 空きマス数を数える
426     int emptyCells = board
427         .expand((row) => row)
428         .where((cell) => cell == 0)
429         .length;
430
431     bool isSolverMode = emptyCells <= 13; // 終盤なら完全読み切りモードへ移行
432     int targetDepth = isSolverMode ? emptyCells : 10; // 終盤は残り手数ま
433     で、それ以外は深さ10で探索
434
435     // 置換表が大きくなりすぎた場合は削除
436     if (_transpositionTable.length > 150000) {
437         _transpositionTable.clear();
438     }
439
440     List<int> bestMove = validMoves.first; // 最善手候補
441
442     // 反復深化探索
443     for (int d = 1; d <= targetDepth; d++) {
444         // Alpha-Beta探索の初期値
445         int alpha = -999999999;
446         int beta = 999999999;
447
448         List<int>? currentBestMove; // この深さでの最善手
449
450         // ヒューリスティックを使って手を並び替える
451         _sortMovesWithHeuristics(validMoves, d, currentPlayer);
452
453         // 各合法手を探索
454         for (int i = 0; i < validMoves.length; i++) {
455             var move = validMoves[i];
456
457             // 元の状態を保存
458             int oldHash = _currentHash;
459             int backupPlayer = currentPlayer;

```

```

459
460 // マス番号へ変換
461 int idx = move[0] * 8 + move[1];
462
463 // Zobrist Hash更新
464 _currentHash ^= _zobristTable[idx][0];
465 _currentHash ^= _zobristTable[idx][currentPlayer];
466
467 // 仮に着手
468 board[move[0]][move[1]] = currentPlayer;
469
470 // 石を反転し、反転した石を保存
471 List<List<int>> flipped = _flipStonesAndReturn(
472     move[0],
473     move[1],
474     currentPlayer,
475 );
476
477 // 次プレイヤーへ変更
478 int nextPlayer = getOpponent(currentPlayer);
479
480 // 手番ハッシュ更新
481 _currentHash ^= _zobristPlayerXor;
482
483 // 相手に合法手があれば手番交代
484 if (hasValidMove(nextPlayer)) {
485     currentPlayer = nextPlayer;
486 }
487
488 int score;
489
490 // 最初の手は通常ウィンドウ探索
491 if (i == 0) {
492     score = -_negascout(d - 1, -beta, -alpha, 60 - d,
isSolverMode);
493 } else {
494     // Null Window Search
495     score = -_negascout(
496         d - 1,
497         -(alpha + 1),
498         -alpha,
499         60 - d,
500         isSolverMode,
501     );
502
503     // Fail-Highなら再探索
504     if (alpha < score && score < beta) {
505         score = -_negascout(d - 1, -beta, -score, 60 - d,
isSolverMode);
506     }
507 }
508
509 // 反転した石を元に戻す
510 for (var stone in flipped) {
511     board[stone[0]][stone[1]] = getOpponent(backupPlayer);
512 }
513
514 // 着手した石を削除
515 board[move[0]][move[1]] = 0;

```

```

516
517     // 手番復元
518     currentPlayer = backupPlayer;
519
520     // ハッシュ値復元
521     _currentHash = oldHash;
522
523     // より良い評価値なら更新
524     if (score > alpha) {
525         alpha = score;
526         currentBestMove = move;
527     }
528 }
529
530 // この深さで最善手が見つければ保存
531 if (currentBestMove != null) {
532     bestMove = currentBestMove;
533 }
534
535 // 勝ち確定レベルなら探索終了
536 if (alpha > 800000) {
537     break;
538 }
539 }
540
541 // 最終的な最善手を返す
542 return bestMove;
543 }
544
545 // 石を反転し、反転した石の位置一覧を返す
546 List<List<int>> _flipStonesAndReturn(int row, int col, int player) {
547     int opponent = getOpponent(player); // 相手プレイヤー番号を取得
548
549     // 反転した石を全て保存するリスト
550     List<List<int>> allFlipped = [];
551
552     // 8方向について調査
553     for (var dir in _directions) {
554         // この方向で反転候補となる石を保存
555         List<List<int>> stonesToFlip = [];
556
557         // 隣接マスから探索開始
558         int r = row + dir[0];
559         int c = col + dir[1];
560
561         // 盤面内である限り探索
562         while (_isInside(r, c)) {
563             // 相手石なら反転候補へ追加
564             if (board[r][c] == opponent) {
565                 stonesToFlip.add([r, c]);
566
567                 // 自分の石に到達した場合
568             } else if (board[r][c] == player) {
569                 // 間にある石を全て反転
570                 for (var stone in stonesToFlip) {
571                     // Zobrist Hash用インデックス
572                     int idx = stone[0] * 8 + stone[1];
573
574                     // ハッシュから相手石状態を除去

```

```

575         _currentHash ^= _zobristTable[idx][opponent];
576
577         // ハッシュへ自分石状態を追加
578         _currentHash ^= _zobristTable[idx][player];
579
580         // 石を反転
581         board[stone[0]][stone[1]] = player;
582
583         // 後で戻せるよう保存
584         allFlipped.add(stone);
585     }
586
587     break;
588 } else {
589     // 空マスなら反転不可
590     break;
591 }
592
593 // 次のマスへ移動
594 r += dir[0];
595 c += dir[1];
596 }
597 }
598
599 return allFlipped; // 反転した石一覧を返す
600 }
601
602 // ≪ NegaScout探索の本体
603 int _negascout(
604     int depth,
605     int alpha,
606     int beta,
607     int ssDepth,
608     bool isSolverMode,
609 ) {
610     int alphaOrig = alpha; // 元のAlpha値を保存
611
612     // 置換表から同一局面を検索
613     TTEEntry? entry = _transpositionTable[_currentHash];
614
615     // 十分な深さで探索済みなら結果を利用
616     if (entry != null && entry.hash == _currentHash && entry.depth >=
depth) {
617         // 完全一致評価値
618         if (entry.flag == 0) {
619             return entry.score;
620
621             // 下限値
622         } else if (entry.flag == 1) {
623             alpha = max(alpha, entry.score);
624
625             // 上限値
626         } else if (entry.flag == 2) {
627             beta = min(beta, entry.score);
628         }
629
630         // 枝刈り成立
631         if (alpha >= beta) {
632             return entry.score;

```

```

633     }
634 }
635
636 // 探索終了条件
637 if (depth == 0 || isGameOver()) {
638     // 終盤完全読みモード
639     if (isSolverMode) {
640         return (countStones(currentPlayer) -
641             countStones(getOpponent(currentPlayer))) *
642             10000;
643     } else {
644         // 評価関数を利用
645         return _evaluateBoardPattern(currentPlayer);
646     }
647 }
648
649 // 合法手一覧を取得
650 List<List<int>> validMoves = _generateValidMoves(currentPlayer);
651
652 // パス処理
653 if (validMoves.isEmpty()) {
654     int nextPlayer = getOpponent(currentPlayer);
655
656     // 両者とも打てない場合は終了
657     if (!hasValidMove(nextPlayer)) {
658         return (countStones(currentPlayer) - countStones(nextPlayer)) *
659             10000;
660     }
661
662     currentPlayer = nextPlayer; // 手番を交代して探索継続
663     _currentHash ^= _zobristPlayerXor; // ハッシュ更新
664
665     int score = -_negascout(depth, -beta, -alpha, ssDepth + 1,
666         isSolverMode);
667
668     // 状態復元
669     _currentHash ^= _zobristPlayerXor;
670     currentPlayer = getOpponent(nextPlayer);
671
672     return score;
673 }
674
675 // 着手順序を最適化
676 _sortNodeMoves(validMoves, ssDepth, entry);
677
678 // この局面での最善手
679 List<int>? bestMoveInThisNode;
680
681 // 全合法手を探索
682 for (int i = 0; i < validMoves.length; i++) {
683     var move = validMoves[i];
684
685     // 元状態保存
686     int oldHash = _currentHash;
687     int backupPlayer = currentPlayer;
688
689     int idx = move[0] * 8 + move[1];
690
691     // 着手によるハッシュ更新

```

```

690     _currentHash ^= _zobristTable[idx][0];
691     _currentHash ^= _zobristTable[idx][currentPlayer];
692
693     // 仮着手
694     board[move[0]][move[1]] = currentPlayer;
695
696     // 石反転
697     List<List<int>> flipped = _flipStonesAndReturn(
698         move[0],
699         move[1],
700         currentPlayer,
701     );
702
703     // 相手手番へ変更
704     int nextPlayer = getOpponent(currentPlayer);
705
706     _currentHash ^= _zobristPlayerXor;
707
708     if (hasValidMove(nextPlayer)) {
709         currentPlayer = nextPlayer;
710     }
711
712     int score;
713
714     // 最初の手は通常探索
715     if (i == 0) {
716         score = -_negascout(
717             depth - 1,
718             -beta,
719             -alpha,
720             ssDepth + 1,
721             isSolverMode,
722         );
723     } else {
724         // Null Window探索
725         score = -_negascout(
726             depth - 1,
727             -(alpha + 1),
728             -alpha,
729             ssDepth + 1,
730             isSolverMode,
731         );
732
733         // 評価値がウィンドウ内なら再探索
734         if (alpha < score && score < beta) {
735             score = -_negascout(
736                 depth - 1,
737                 -beta,
738                 -score,
739                 ssDepth + 1,
740                 isSolverMode,
741             );
742         }
743     }
744
745     // 反転石を元に戻す
746     for (var stone in flipped) {
747         board[stone[0]][stone[1]] = getOpponent(backupPlayer);

```

```

748     }
749
750     // 着手を取り消す
751     board[move[0]][move[1]] = 0;
752
753     // 手番復元
754     currentPlayer = backupPlayer;
755
756     // ハッシュ復元
757     _currentHash = oldHash;
758
759     // Beta Cut発生
760     if (score >= beta) {
761         // Killer Move登録
762         if (ssDepth < 60 && _killerMoves[ssDepth][0] != move) {
763             _killerMoves[ssDepth][1] = _killerMoves[ssDepth][0];
764             _killerMoves[ssDepth][0] = move;
765         }
766
767         // History Heuristic更新
768         _historyTable[move[0]][move[1]] += depth * depth;
769
770         // 置換表へ保存
771         _transpositionTable[_currentHash] = TTEEntry(
772             hash: _currentHash,
773             score: beta,
774             depth: depth,
775             flag: 1,
776             bestMove: move,
777         );
778
779         return beta;
780     }
781
782     // より良い手なら更新
783     if (score > alpha) {
784         alpha = score;
785         bestMoveInThisNode = move;
786     }
787 }
788
789 // 保存する評価種別を決定
790 int flag = (alpha <= alphaOrig) ? 2 : 0;
791
792 // 置換表へ保存
793 _transpositionTable[_currentHash] = TTEEntry(
794     hash: _currentHash,
795     score: alpha,
796     depth: depth,
797     flag: flag,
798     bestMove: bestMoveInThisNode,
799 );
800
801 // 最終評価値を返す
802 return alpha;
803 }
804
805 // 盤面評価関数
806 int _evaluateBoardPattern(int player) {

```

```

807     int opponent = getOpponent(player); // 相手プレイヤー番号取得
808     int patternScore = 0; // パターン評価値
809
810     // 上辺評価
811     patternScore += _evaluateEdgePattern(0, 0, 0, 1, 1, 1, player);
812
813     // 下辺評価
814     patternScore += _evaluateEdgePattern(7, 0, 0, 1, 6, 1, player);
815
816     // 左辺評価
817     patternScore += _evaluateEdgePattern(0, 0, 1, 0, 1, 1, player);
818
819     // 右辺評価
820     patternScore += _evaluateEdgePattern(0, 7, 1, 0, 1, 6, player);
821
822     // 安定石判定結果取得
823     var stableMatrix = _calculateCompleteStableStones();
824
825     int myStables = 0;
826     int oppStables = 0;
827
828     // 安定石数を集計
829     for (int r = 0; r < 8; r++) {
830         for (int c = 0; c < 8; c++) {
831             if (stableMatrix[r][c] == player) {
832                 myStables++;
833             } else if (stableMatrix[r][c] == opponent) {
834                 oppStables++;
835             }
836         }
837     }
838
839     int myFrontier = 0;
840     int oppFrontier = 0;
841
842     // フロンティア石数を計算
843     for (int r = 0; r < 8; r++) {
844         for (int c = 0; c < 8; c++) {
845             // 空マスは評価対象外
846             if (board[r][c] == 0) {
847                 continue;
848             }
849
850             bool isFrontier = false;
851
852             // 周囲8方向を調べる
853             for (var dir in _directions) {
854                 int nr = r + dir[0]; // 隣接マスの行
855                 int nc = c + dir[1]; // 隣接マスの列
856
857                 // 隣に空マスがあればフロンティア石
858                 if (_isInside(nr, nc) && board[nr][nc] == 0) {
859                     isFrontier = true;
860                     break;
861                 }
862             }
863
864             // フロンティア石を自分・相手別に集計
865             if (isFrontier) {

```

```

866         if (board[r][c] == player) {
867             myFrontier++;
868         } else {
869             oppFrontier++;
870         }
871     }
872 }
873 }
874
875 int myMobilityEstimate = 0;
876 int oppMobilityEstimate = 0;
877
878 // 行動可能性（可動性）を推定
879 for (int r = 0; r < 8; r++) {
880     for (int c = 0; c < 8; c++) {
881         // 空マスのみを対象に調査
882         if (board[r][c] == 0) {
883             // 空マスの周囲8方向を確認
884             for (var d in _directions) {
885                 int nr = r + d[0]; // 隣接行
886                 int nc = c + d[1]; // 隣接列
887
888                 if (_isInside(nr, nc)) {
889                     // 相手石の隣に空マスがあるほど自分の手候補が増える
890                     if (board[nr][nc] == opponent) {
891                         myMobilityEstimate++;
892                     }
893
894                     // 自分石の隣に空マスがあるほど相手の手候補が増える
895                     if (board[nr][nc] == player) {
896                         oppMobilityEstimate++;
897                     }
898                 }
899             }
900         }
901     }
902 }
903
904 // 可動性評価値を算出
905 int mobilityScore = (myMobilityEstimate - oppMobilityEstimate) * 15;
906
907 // フロンティア石評価値を算出（少ない方が有利）
908 int frontierScore = (oppFrontier - myFrontier) * 18;
909
910 // 安定石評価値を算出（多い方が有利）
911 int stableScore = (myStables - oppStables) * 450;
912
913 // 各評価値を合計して盤面評価値を返す
914 return patternScore + mobilityScore + frontierScore + stableScore;
915 }
916
917 // 辺パターンの評価値を取得
918 int _evaluateEdgePattern(
919     int startR,
920     int startC,
921     int dr,
922     int dc,
923     int x1R,

```

```

924     int x1C,
925     int player,
926 ) {
927     int opponent = getOpponent(player); // 相手プレイヤー取得
928     int index = 0; // パターンテーブル参照用インデックス
929
930     // 辺の8マスをも3進数としてエンコード
931     for (int i = 0; i < 8; i++) {
932         int cell = board[startR + i * dr][startC + i * dc];
933
934         // 自分=1, 相手=2, 空=0 に変換
935         int val = (cell == player) ? 1 : ((cell == opponent) ? 2 : 0);
936
937         index += val * _pow3[i];
938     }
939
940     // Xスクエア1を追加
941     int cellX1 = board[x1R][x1C];
942     int valX1 = (cellX1 == player) ? 1 : ((cellX1 == opponent) ? 2 : 0);
943     index += valX1 * _pow3[8];
944
945     // Xスクエア2の座標を計算
946     int x2R = x1R + (dr != 0 ? 5 * dr : 0);
947     int x2C = x1C + (dc != 0 ? 5 * dc : 0);
948
949     // Xスクエア2を追加
950     int cellX2 = board[x2R][x2C];
951     int valX2 = (cellX2 == player) ? 1 : ((cellX2 == opponent) ? 2 : 0);
952     index += valX2 * _pow3[9];
953
954     // 事前計算済みパターン評価値を返す
955     return _edgePatternTable[index];
956 }
957
958 // 完全安定石を計算
959 List<List<int>> _calculateCompleteStableStones() {
960     // 安定石管理用の盤面
961     List<List<int>> stable = List.generate(8, (_) => List.filled(8, 0));
962
963     bool changed = true; // 安定石が増えたかを記録
964
965     // 左上角が埋まっていれば安定石
966     if (board[0][0] != 0) {
967         stable[0][0] = board[0][0];
968     }
969
970     // 右上角が埋まっていれば安定石
971     if (board[0][7] != 0) {
972         stable[0][7] = board[0][7];
973     }
974
975     // 左下角が埋まっていれば安定石
976     if (board[7][0] != 0) {
977         stable[7][0] = board[7][0];
978     }
979
980     // 右下角が埋まっていれば安定石
981     if (board[7][7] != 0) {
982         stable[7][7] = board[7][7];

```

```

983     }
984
985     // 安定石が増えなくなるまで繰り返す
986     while (changed) {
987         changed = false;
988
989         for (int r = 0; r < 8; r++) {
990             for (int c = 0; c < 8; c++) {
991                 // 空マスまたは既に安定石ならスキップ
992                 if (board[r][c] == 0 || stable[r][c] != 0) {
993                     continue;
994                 }
995
996                 // 横方向の安定判定
997                 bool stableHorizontal = _isAxisStable(r, c, 0, 1, stable);
998
999                 // 縦方向の安定判定
1000                 bool stableVertical = _isAxisStable(r, c, 1, 0, stable);
1001
1002                 // 斜め（\）方向の安定判定
1003                 bool stableDiag1 = _isAxisStable(r, c, 1, 1, stable);
1004
1005                 // 斜め（／）方向の安定判定
1006                 bool stableDiag2 = _isAxisStable(r, c, 1, -1, stable);
1007
1008                 // 全方向で安定なら完全安定石とする
1009                 if (stableHorizontal &&
1010                     stableVertical &&
1011                     stableDiag1 &&
1012                     stableDiag2) {
1013                     stable[r][c] = board[r][c];
1014                     changed = true;
1015                 }
1016             }
1017         }
1018     }
1019
1020     // 完成した安定石情報を返す
1021     return stable;
1022 }
1023
1024 // 指定方向において石が安定石か判定
1025 bool _isAxisStable(int r, int c, int dr, int dc, List<List<int>>
stable) {
1026     int color = board[r][c]; // 対象石の色
1027
1028     bool side1Stable = false;
1029
1030     // 正方向の隣接マスを取得
1031     int nr = r + dr;
1032     int nc = c + dc;
1033
1034     // 盤外なら端に接しているため安定
1035     if (!_isInside(nr, nc)) {
1036         side1Stable = true;
1037
1038         // 同色の安定石に接している場合も安定
1039     } else if (stable[nr][nc] == color) {
1040         side1Stable = true;

```

```

1041     }
1042
1043     bool side2Stable = false;
1044
1045     // 逆方向の隣接マスを取得
1046     nr = r - dr;
1047     nc = c - dc;
1048
1049     // 盤外なら端に接しているため安定
1050     if (!_isInside(nr, nc)) {
1051         side2Stable = true;
1052
1053         // 同色の安定石に接している場合も安定
1054     } else if (stable[nr][nc] == color) {
1055         side2Stable = true;
1056     }
1057
1058     // 両方向が安定している場合のみ安定石と判定
1059     return side1Stable && side2Stable;
1060 }
1061
1062 // ルート探索時の手順序付け
1063 void _sortMovesWithHeuristics(List<List<int>> moves, int depth, int
player) {
1064     // History Heuristicと静的位置評価を利用してソート
1065     moves.sort((a, b) {
1066         // 手Aの評価値
1067         int scoreA = _historyTable[a[0]][a[1]] + _getStaticWeight(a[0],
a[1]);
1068
1069         // 手Bの評価値
1070         int scoreB = _historyTable[b[0]][b[1]] + _getStaticWeight(b[0],
b[1]);
1071
1072         // 高評価の手を先頭へ配置
1073         return scoreB.compareTo(scoreA);
1074     });
1075 }
1076
1077 // NegaScout探索中の手順序付け
1078 void _sortNodeMoves(List<List<int>> moves, int ssDepth, TTEntry?
entry) {
1079     // 置換表から最善手候補を取得
1080     List<int>? ttMove = entry?.bestMove;
1081
1082     // Killer Moveを取得
1083     List<int>? killer1 = ssDepth < 60 ? _killerMoves[ssDepth][0] : null;
1084
1085     List<int>? killer2 = ssDepth < 60 ? _killerMoves[ssDepth][1] : null;
1086
1087     moves.sort((a, b) {
1088         int rankA = 0;
1089         int rankB = 0;
1090
1091         // 置換表の最善手を最優先
1092         if (ttMove != null && a[0] == ttMove[0] && a[1] == ttMove[1]) {
1093             rankA = 100000;
1094         }
1095

```

```

1096     if (ttMove != null && b[0] == ttMove[0] && b[1] == ttMove[1]) {
1097         rankB = 100000;
1098     }
1099
1100     // 第一Killer Moveを優先
1101     if (killer1 != null && a[0] == killer1[0] && a[1] == killer1[1]) {
1102         rankA = max(rankA, 50000);
1103     }
1104
1105     if (killer1 != null && b[0] == killer1[0] && b[1] == killer1[1]) {
1106         rankB = max(rankB, 50000);
1107     }
1108
1109     // 第二Killer Moveを優先
1110     if (killer2 != null && a[0] == killer2[0] && a[1] == killer2[1]) {
1111         rankA = max(rankA, 25000);
1112     }
1113
1114     if (killer2 != null && b[0] == killer2[0] && b[1] == killer2[1]) {
1115         rankB = max(rankB, 25000);
1116     }
1117
1118     // ランクが同じ場合はHistory Heuristicで比較
1119     if (rankA == rankB) {
1120         return _historyTable[b[0]][b[1]].compareTo(_historyTable[a[0]]
[a[1]]);
1121     }
1122
1123     // 高ランク順に並べる
1124     return rankB.compareTo(rankA);
1125 });
1126 }
1127
1128 // 盤面位置ごとの固定評価値を取得
1129 int _getStaticWeight(int r, int c) {
1130     // リバーシ定番の重みテーブル
1131     const List<List<int>> weightMap = [
1132         // 角は非常に高評価
1133         [200, -80, 20, 5, 5, 20, -80, 200],
1134
1135         // 角の隣 (X・Cマス) は危険なため低評価
1136         [-80, -120, -5, -5, -5, -5, -120, -80],
1137
1138         [20, -5, 15, 3, 3, 15, -5, 20],
1139
1140         [5, -5, 3, 3, 3, 3, -5, 5],
1141
1142         [5, -5, 3, 3, 3, 3, -5, 5],
1143
1144         [20, -5, 15, 3, 3, 15, -5, 20],
1145
1146         [-80, -120, -5, -5, -5, -5, -120, -80],
1147
1148         [200, -80, 20, 5, 5, 20, -80, 200],
1149     ];
1150
1151     // 指定座標の評価値を返す
1152     return weightMap[r][c];
1153 }
1154

```

```
1155 // 3進数エンコード用テーブル
1156 static const List<int> _pow3 = [1, 3, 9, 27, 81, 243, 729, 2187, 6561,
1157 19683];
}
```